Lezione 2; Applicazioni Web, Caratteristiche di un’applicazione a servizi, Frontend vs. Backend, il ruolo del Browser e del Server Web, differenza tra siti Web Statici e Applicazioni Dinamiche

Introduzione

Nella scorsa lezione abbiamo affrontato i fondamenti delle applicazioni web, ora vediamo cosa sono questi applicazioni e come si sviluppano.

Cos'è un'applicazione web

Un’**applicazione web (web app)** è un programma software accessibile tramite un **browser** (Chrome, Firefox, Safari, Edge, ecc.) e fruibile attraverso una rete, tipicamente **Internet** o, in ambito aziendale, una **intranet**.

La differenza principale rispetto a un’applicazione desktop tradizionale è **dove viene eseguito il codice**:

- **Un’app desktop viene installata ed eseguita interamente sul computer dell’utente.**
- Una web app, invece, è **distribuita**:
  - Il **backend**:
    - **(la logica, la gestione dei dati, le operazioni pesanti) risiede su un server remoto.**
  - Il **frontend**:
    - **(l’interfaccia grafica con cui interagisce l’utente) gira nel browser**.

La caratteristica distintiva: l'interattività

Un sito web tradizionale può limitarsi a mostrare informazioni statiche, come una pagina di sola lettura.
Una web app, invece, permette all’utente di **interagire**, compiere azioni e manipolare dati in tempo reale.

Esempi comuni:

- **Gmail** (gestione email online)

- **Google Docs** (documenti collaborativi in cloud)
- **Facebook** (social network interattivo)
- **Amazon** (e-commerce con carrello, pagamenti, ordini)
- **Trello** (gestione progetti e task management)

Caratteristiche principali di una Web App

1. **Nessuna installazione richiesta → basta un browser.**
2. **Accessibilità universale → funziona da qualsiasi dispositivo connesso a Internet (PC, smartphone, tablet).**
3. **Aggiornamenti centralizzati → quando viene aggiornata sul server, tutti gli utenti vedono subito la nuova versione, senza bisogno di reinstallare nulla.**

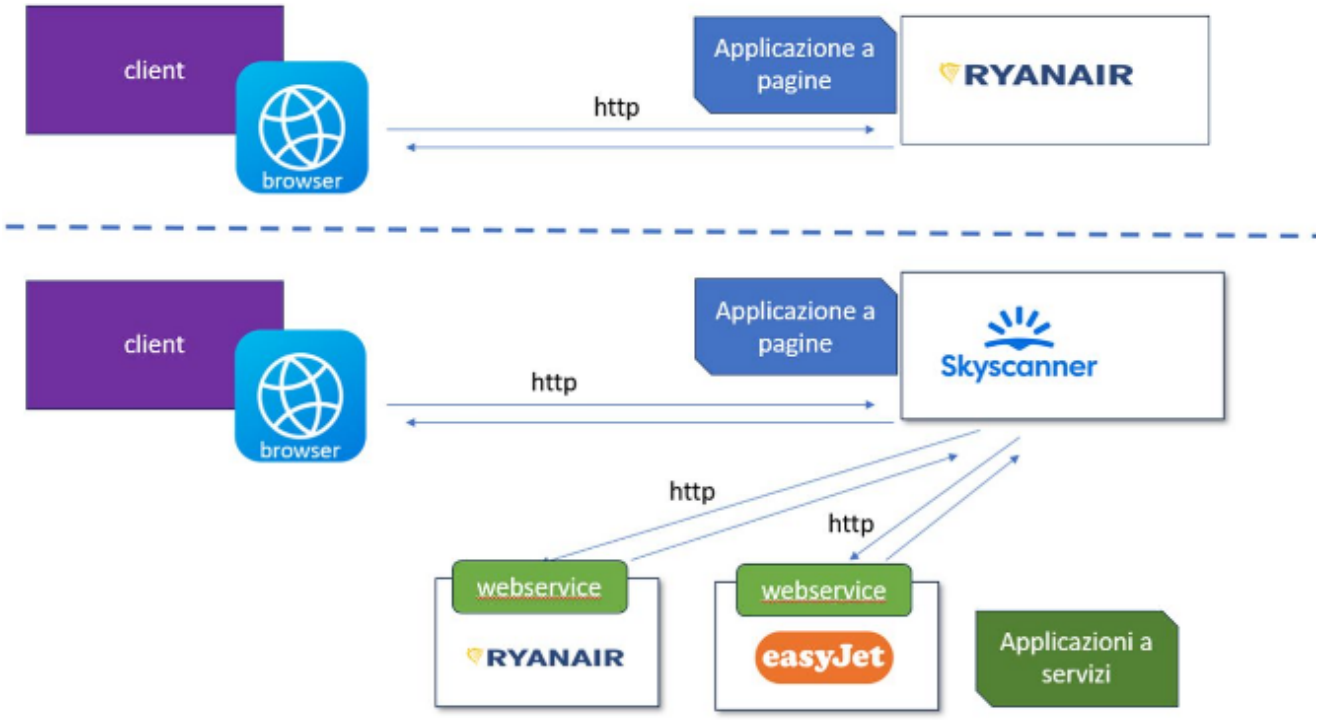
Le applicazioni web

La maggior parte delle applicazioni web utilizzano l'architettura client - server
Le caratteristiche di queste applicazioni sono:

1. Utilizzano il protocollo TCP/IP
2. Si dividono in 2 famiglie principali:
   - **Applicazioni a pagine:**
     - **il client è l'utente che, attraverso il browser, chiede un servizio al server e riceve in risposta una pagina web.**
   - **Applicazioni a servizi:**
     - **il client è un programma che chiede un servizio al server e riceve in risposta dati.**

Spiegazione delle applicazioni web

Per comprendere meglio come queste 2 tipologie lavorano prendiamo ad esempio questa immagine



Applicazioni web.png

Parte alta dell'immagine: Applicazioni a pagine (esempio Ryanair)

- **Client → l'utente con il suo browser (Chrome, Firefox, ecc.).**
- **L'utente invia una** richiesta HTTP **al server di Ryanair (es. chiede i voli disponibili).**
- **Il server Ryanair elabora la richiesta e risponde con una pagina HTML già pronta.**
- Quindi il browser mostra quella pagina all'utente.

> 📋 **Questo modello è detto applicazione a pagine, perché la comunicazione tra client e server avviene tramite pagine complete (HTML).**
>
> L'utente riceve interfacce grafiche già pronte, ma non dati "puri".

Parte bassa: Applicazioni a servizi (Esempio Skyscanner)

Qui la logica è più complessa:

- **L'utente accede a Skyscanner tramite il browser.**
- **Skyscanner non ha un suo database di voli: per rispondere alle richieste degli utenti, interroga i web service delle varie compagnie aeree (Ryanair, EasyJet, ecc.).**

- **Ogni compagnia fornisce un servizio via API (Application Programming Interface), cioè non restituisce una pagina HTML, ma dati grezzi in formato strutturato (tipicamente JSON o XML).**
- **Skyscanner raccoglie i dati da Ryanair, EasyJet e altre compagnie, li elabora e poi costruisce la pagina che l'utente vede nel browser.**

Questo modello è detto **applicazione a servizi**, perché il "client" non è un utente umano che riceve pagine già pronte, ma è un **programma** che dialoga con altri server e scambia **dati**.

Differenza chiave

- **Applicazioni a pagine**: il browser dell'utente chiede → il server risponde con una pagina HTML.
- **Applicazioni a servizi**: un'applicazione (come Skyscanner) fa da intermediario → interroga più web service → riceve dati grezzi → li aggrega e li presenta all'utente in una pagina.

Il client come programma

Quindi come abbiamo visto nelle applicazioni a servizi, il **client non è più solo un utente umano che naviga da browser**, ma può essere un vero e proprio **programma software** che dialoga con un server.

Esempi di client:

- **Un'app Android o iOS**, che invia richieste a un server remoto per recuperare dati (es. un'app meteo che chiede le previsioni).
- **Un browser** che, all'interno di una pagina web, esegue codice **JavaScript** capace di fare richieste AJAX o fetch verso API.

In questo scenario, **client e server si scambiano esclusivamente dati**, non pagine HTML. Questi dati sono di solito in formato **JSON** o **XML**, così da poter essere facilmente interpretati da programmi diversi.

Interoperabilità del sistema Client-Server

Un vantaggio fondamentale di questo modello è che è **interoperabile**:

✓ **In che senso è interoperabile?**

- **Il client e il server non devono essere scritti nello stesso linguaggio di programmazione o sulla stessa infrastruttura.**
- Ad esempio, **un'app mobile in Kotlin (Android) o Swift (iOS) può comunicare con un server sviluppato in Node.js, Java, Python o qualsiasi altra tecnologia, purché entrambi rispettino lo stesso protocollo di comunicazione (tipicamente HTTP)**.

Questa indipendenza permette di integrare sistemi eterogenei e di farli collaborare senza vincoli tecnologici.

Stili di Realizzazione: SOAP e REST

La comunicazione Client-Server per applicazioni a servizi segue solitamente due principali stili architetturali:

1. **SOAP (Simple Object Access Protocol)** – lo stile classico
   - **Basato su XML.**
   - **Più "verboso" e rigido, con regole precise su formati e protocolli.**
   - **Ancora usato in ambiti entreprise e in contesti che richiedono molta formalità (es. banche, assicurazioni).**
2. **REST (Representational State Transfer)** – **lo stile moderno**
   - **Basato su** HTTP standard**.**
   - **Usa formati leggeri e semplici, come** JSON**.**
   - **Più flessibile, veloce e facile da integrare, oggi è lo standard de facto per le** API web**.**

Frontend vs. Backend nella applicazioni web

Nelle architetture **Client-Server**, le applicazioni web si dividono in due aree principali di sviluppo:

1. **Frontend** (lato client)
2. **Backend** (lato server).
   Queste due componenti collaborano per fornire all'utente un'esperienza completa, combinando interfaccia visiva e logica applicativa.

Frontend (Lato Client)

Il Frontend è tutto ciò che l'utente vede e con cui interagisce nel browser.
Si può paragonare alla **parte visibile dell'iceberg**:

- quello che appare davanti ai suoi occhi.

Ruolo principale:

- **Creare l'interfaccia utente (UI)**
- **assicurare una buona esperienza utente (UX).**

Responsabilità principali:

- **Struttura dei contenuti: organizzare testi, immagini, tabelle, pulsanti e form.**
- **Stile e layout: curare colori, font, posizionamento e adattabilità ai diversi dispositivi (responsive design).**
- **Interattività: gestire click, animazioni, aggiornamenti dinamici della pagina senza ricaricare completamente il browser (es. usando AJAX o fetch).**

Tecnologie principali:

- **HTML (HyperText Markup Language): definisce la struttura dei contenuti.**
- **CSS (Cascading Style Sheets): cura l'aspetto grafico e il layout.**
- **JavaScript: permette di aggiungere dinamismo e interattività.**
  - **Framework moderni come React, Angular e Vue.js semplificano la creazione di interfacce complesse e reattive.**

🔗 Collegamento con le applicazioni web

Nel modello delle **applicazioni a pagine:**
il frontend riceve direttamente HTML pronto dal server.
Nel modello delle **applicazioni a servizi:**
il frontend può invece essere un programma che richiede **dati grezzi (JSON/XML)** al backend e costruisce la pagina dinamicamente.

Backend

Il **Backend** è il "cervello" dell'applicazione, **la parte invisibile all'utente, o la porzione sommersa dell'iceberg.**

Ruolo principale: **gestire logica applicativa, dati e sicurezza.**

Responsabilità principali:

- **Logica applicativa:** **eseguire operazioni principali dell'applicazione, come elaborare pagamenti o registrare utenti.**
- **Gestione del database:** **creare, leggere, aggiornare e cancellare dati (es. prodotti, post di blog, informazioni degli utenti).**
- **Autenticazione e autorizzazione:** **verificare chi è l'utente e cosa può fare.**
- **Esposizione di API:** **creare endpoint che il frontend può interrogare per leggere o scrivere dati.**

Tecnologie principali:

- **Linguaggi e framework:** Python (Django, Flask), JavaScript (Node.js), Java (Spring), PHP (Laravel), C# (.NET).
- **Database:** SQL (PostgreSQL, MySQL), NoSQL (MongoDB, Redis).
- **Web server:** Apache, Nginx.

🔗 **Collegamento con le applicazioni web:**

- Nelle **applicazioni a pagine**, **il server restituisce direttamente pagine HTML pronte.**
- Nelle **applicazioni a servizi**, **il backend fornisce dati grezzi tramite API, che il frontend usa per costruire dinamicamente le interfacce.**

🔗

Il ponte tra Frontend e Backend: le API

Le API (Application Programming Interface) sono il canale di comunicazione tra frontend e backend.

- **Permettono al frontend di ottenere o inviare dati senza accedere direttamente al database.**
- Rendono possibile l'interoperabilità: **frontend e backend possono essere sviluppati con linguaggi diversi e ospitati su infrastrutture differenti, purché rispettino lo stesso protocollo di comunicazione (tipicamente HTTP/HTTPS).**

Esempi di client:

app mobile Android/iOS, browser con JavaScript che fa fetch o AJAX verso API.

Esempi di server:

backend in Python, Node.js, Java, ecc. che risponde a richieste HTTP con dati JSON/XML.

Il ruolo del browser web e server web

Nell'architettura **Client-Server**, oltre all'utente e alla logica applicativa, esistono due attori software fondamentali:

1. **il browser web** (il client software)
2. **il server web** (il backend software).
   Questi due componenti operano insieme per gestire le richieste e le risposte tra frontend e backend.

1. Browser web (Client Software)

Il **browser** è il software che funge da lato client dell'architettura:

- **rappresenta il "volto visibile" dell'applicazione per l'utente.**

Funzioni principali:

1. **Invio delle richieste:**
   - **ogni volta che l'utente digita un URL o clicca su un link, il browser invia una richiesta HTTP al server per ottenere la risorsa desiderata.**
2. **Ricezione e interpretazione della risposta:**
   - **il browser riceve dal server codice HTML, CSS e JavaScript. Il suo compito è renderizzare questi contenuti, trasformandoli in una pagina web completa, visivamente corretta e interattiva.**
3. **Esecuzione del codice JavaScript:**
   - **gestisce la logica di interattività definita dal frontend, come animazioni, aggiornamenti dinamici della pagina e richieste AJAX o**

fetch verso le API del backend.

🔗 **Collegamento con la sezione precedente:**

il browser è la parte **frontend** che trasforma i dati ricevuti dal server in un'interfaccia fruibile dall'utente, sia che si tratti di pagine già pronte (modello a pagine) sia di dati grezzi da elaborare (modello a servizi).

2. Server Web (Backend Software)

Il server web è il software che gestisce le richieste sul lato server.
Esempi comuni sono **Apache** o **Nginx**, che girano su macchine server dedicate.

Funzioni principali:

1. **Ascolto delle richieste:**
   - **il server è sempre in ascolto delle richieste HTTP in arrivo dai browser.**
2. **Gestione dei file statici:**
   - **se la richiesta riguarda un file statico (immagine, foglio di stile CSS, script JS), il server lo recupera e lo invia direttamente al browser.**
3. **Gestione delle risorse dinamiche:**
   - **se la richiesta riguarda una risorsa dinamica (ad esempio `/api/products/123`), il server inoltra la richiesta all'applicazione backend vera e propria (scritta in Python, Java, Node.js, ecc.).**
   - **L'applicazione elabora la richiesta, genera i dati di risposta (spesso in JSON) e li restituisce al server, che a sua volta li invia al browser del client.**

🔗 **Collegamento con la sezione precedente:**

il server web rappresenta il **backend**, gestendo la logica applicativa, la sicurezza e l'accesso ai dati.
Attraverso le API, fornisce al frontend le informazioni necessarie per costruire le interfacce dinamiche viste dall'utente.

📋 **In sintesi**

browser e server web collaborano strettamente:

- il browser invia richieste e mostra contenuti.
- il server elabora dati e risponde correttamente alle richieste, rispettando i ruoli definiti per frontend e backend.
  Questa collaborazione costituisce la base del funzionamento di ogni applicazione web moderna, sia essa a pagine o a servizi.

Differenza tra siti Web statici e Applicazioni Dinamiche

La distinzione tra **siti web statici** e **applicazioni web dinamiche** è fondamentale per capire perché molte applicazioni moderne richiedono un backend complesso e l'uso di API.

Sito web statico

Un sito web statico è costituito da file già pronti: HTML, CSS e JavaScript pre-costruiti.

Caratteristiche principali:

- **Quando un utente richiede una pagina, il server web recupera il file corrispondente e lo invia al browser così com'è, senza elaborazioni aggiuntive.**
- **Il contenuto non cambia in base all'utente o alle sue interazioni: ogni visitatore vede esattamente la stessa pagina.**
- **Analogia:** una brochure o un volantino digitale.
- **Uso ideale:** siti di presentazione, portfolio, documentazione, landing page. Questi siti **non richiedono un backend complesso** né un database.

🔗 Collegamento con il Frontend/Backend:

il browser mostra direttamente i contenuti inviati dal server web; non c'è logica dinamica né interazione personalizzata, quindi il backend rimane minimale.

Applicazione Web Dinamica

Un'applicazione web dinamica genera pagine o dati "al volo", rispondendo alle richieste dell'utente in modo personalizzato.

Caratteristiche principali:

- **Quando un utente richiede una risorsa, la richiesta passa al backend, che esegue la logica applicativa.**
- **Il backend può interrogare un database, personalizzare i contenuti in base all'utente (es. "Ciao, Marco!") e costruire la risposta, che può essere HTML già pronto o dati strutturati (JSON/XML).**
- **Il contenuto è quindi dinamico e interattivo.**
- **Analogia: una conversazione: la risposta dipende da chi sei e da cosa chiedi.**
- **Uso ideale:** social network, e-commerce, piattaforme di online banking, dashboard e qualsiasi applicazione che richieda personalizzazione e interattività.

🔗 Collegamento con le API:

nelle applicazioni moderne, le API REST permettono al frontend di richiedere e manipolare i dati in modo strutturato e prevedibile.
Il backend elabora la richiesta, interroga il database se necessario, e restituisce la risposta al client, garantendo interoperabilità e flessibilità.